

C 언어 EXPRESS(개정4판)



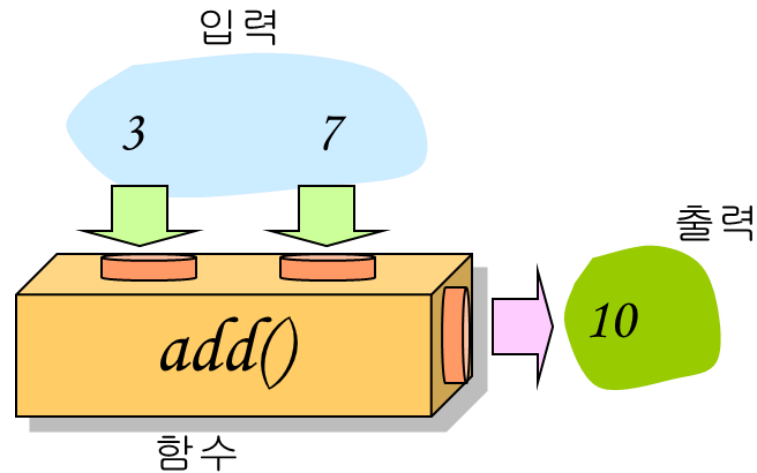
제 8 장 함수



함수의 특징

2

- 함수는 특정한 작업을 수행하기 위한 명령어들의 모음이다.
- 함수는 서로 구별되는 이름을 가지고 있다.
- 함수는 특정한 작업을 수행한다.
- 함수는 입력을 받을 수 있고 결과를 반환할 수 있다.



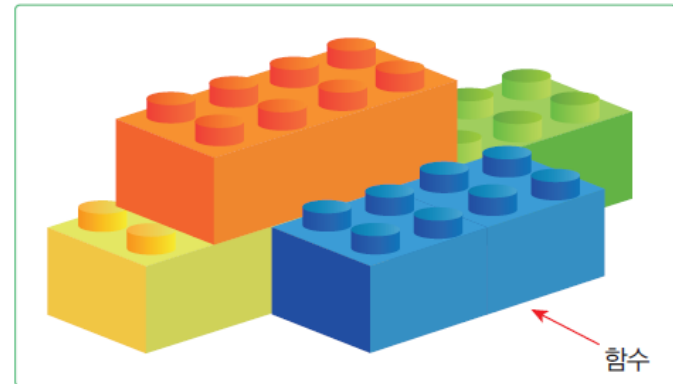


함수의 장점

3

- 함수를 사용하면 코드가 중복되는 것을 막을 수 있다.
- 한번 작성된 함수는 여러 번 재사용할 수 있다.
- 함수를 사용하면 전체 프로그램을 모듈로 나눌 수 있어서 개발 과정이 쉬워지고 보다 체계적이 되면서 유지보수도 쉬워진다.

프로그램





함수의 종류

4

함수 { 사용자 정의 함수
라이브러리 함수



내가 원하는 함수가 없으니
직접 만들어야지!



이 함수들은 기본적으로
제공됩니다.



함수의 정의

5

Syntax

함수 정의

예

반환형 함수 이름 매개 변수 (현재는 없다)

```
void print_stars()
```

```
{
```

```
    for(int i=0; i<30; i++)
```

```
        printf("*");
```

```
}
```

함수 몸체



함수 정의

6

- 반환형

- 반환형은 함수가 처리를 종료한 후에 호출한 곳으로 반환하는 데이터의 유형을 말한다.

- 함수 이름

- 함수 이름은 식별자에 대한 규칙만 따른다면 어떤 이름이라도 가능하다.
- 함수의 기능을 암시하는 (동사+명사)를 사용하면 좋다.

```
int    square()  
double compute_average()  
void   set_cursor_type()
```

// 정수를 제공하는 함수
// 평균을 구하는 함수
// 커서의 타입을 설정하는 함수

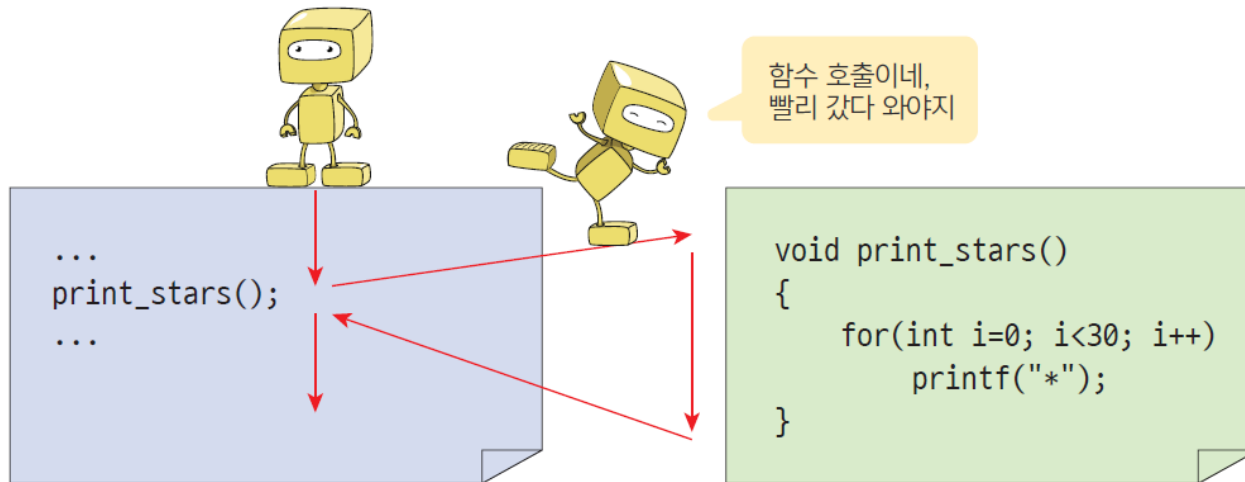
함수 이름



함수 호출

7

- 함수 호출(function call)이란 `print_stars()`와 같이 함수의 이름을 써주는 것이다.
- 함수 안의 문장들은 호출되기 전까지는 전혀 실행되지 않는다. 함수를 호출하게 되면 현재 실행하고 있는 코드는 잠시 중단되고, 호출된 함수로 이동하여서 함수 몸체 안의 문장들이 순차적으로 실행된다.
- 호출된 함수의 실행이 끝나면 호출한 위치로 되돌아가서 잠시 중단되었던 코드가 실행을 재개한다.





매개 변수와 반환값

8

Syntax

함수의 구조

예

```
반환형   함수 이름   매개 변수
int max(int x, int y)
{
    if( x > y )
        return x;
    else
        return y;
}
```

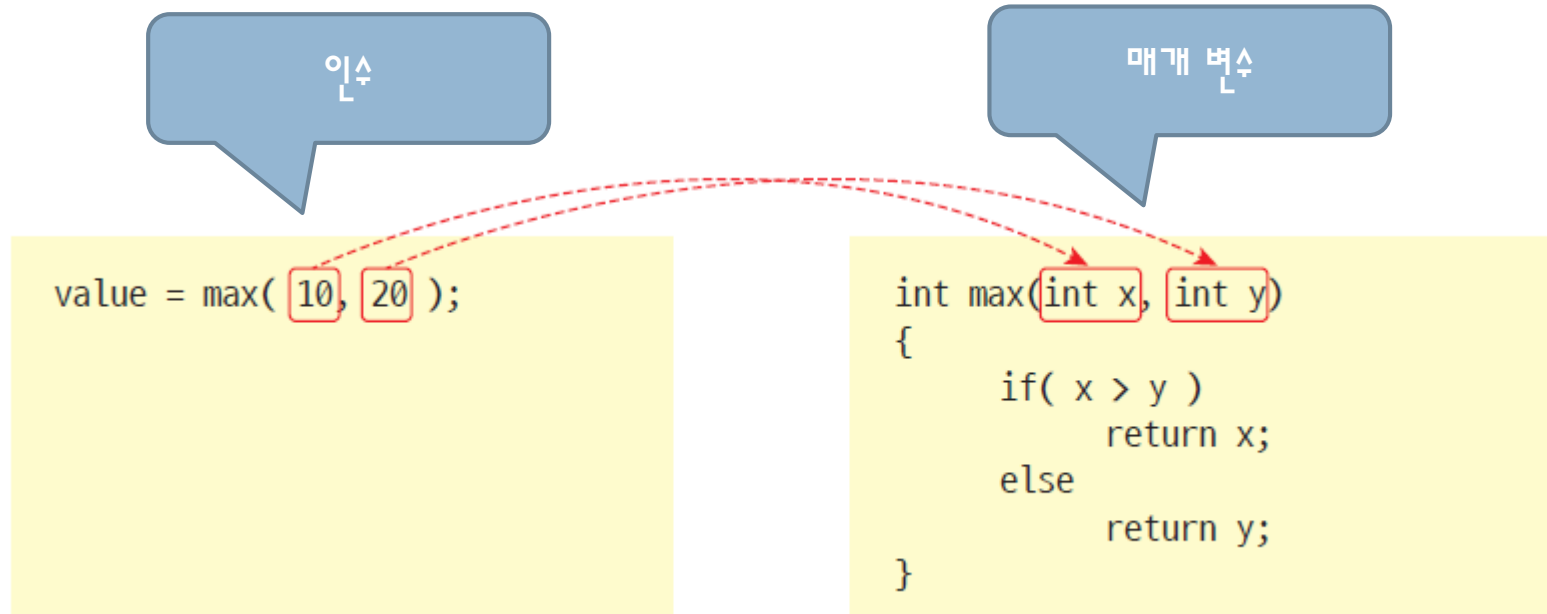
함수 몸체



인수와 매개변수

9

- 인수(argument)는 호출 프로그램에 의하여 함수에 실제로 전달되는 값이다.
- 매개 변수(parameter)는 이 값을 전달받는 변수이다.





인수와 매개 변수

10

- 만약 매개 변수가 없는 경우에는 `print_stars(void)`와 같이 매개변수 위치에 `void`를 써주거나 `print_stars()`와 같이 아무 것도 적지 않으면 된다.
- 함수가 호출될 때마다 인수는 달라질 수 있다.
- 매개 변수의 개수는 정확히 일치하여야 한다.

```
max(10);      // max() 함수를 호출할 때는 인수가 두개이어야 한다.  
max( );      // max() 함수를 호출할 때는 인수가 두개이어야 한다.
```



- 반환값(return value)은 함수가 호출한 곳으로 반환하는 작업의 결과값이다.
- 값을 반환하려면 **return** 문장 다음에 수식을 써주면 수식의 값이 반환된다.
- 인수는 여러 개가 있을 수 있으나 반환값은 하나만 가능하다.

```
value = max( 10, 20 );
```

```
int max(int x, int y)
{
    if( x > y )
        return x;
    else
        return y;
}
```





예제

12

- 위에서 작성한 `max()` 함수를 호출하여서 사용자가 입력한 값 중에서 더 큰 값을 찾아보자.

정수 2개를 입력하시오: 10 20
더 큰 값은 20입니다.



예제

13

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
```

```
int max(int x, int y)
{
    if (x > y)
        return x;
    else
        return y;
}
```

```
int main(void)
{
    int x, y, larger;

    printf("정수 2개를 입력하시오: ");
    scanf("%d %d", &x, &y);

    larger = max(x, y);
    printf("더 큰 값은 %d입니다. \n", larger);
    return 0;
}
```



함수 연형

14

- 아래의 코드를 컴파일하면 오류가 발생한다.

```
#include <stdio.h>

int main(void)
{
    printf("섭씨 %lf도는 화씨 %lf입니다. \n", 36.0, c_to_f(36.0));
    return 0;
}

double c_to_f(double c_temp)
{
    return 9.0 / 5.0 * c_temp + 32;
}
```

전체 솔루션 1 오류 2 경고 0 메시지 빌드 + IntelliSense 검색 오류 목록

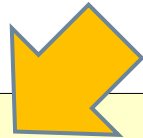
코드	설명	프로젝트	파일	줄	Suppress
C4013	'c_to_f'(가) 정의되지 않았습니다. extern은 int형을 반환하는 것으로 간주합니다.	ConsoleApplication3	test.c	5	
C4477	'printf': 서식 문자열 '%lf'에 'double' 형식의 인수가 필요하지만 variadic 인수 2의 형식이 'int'입니다.	ConsoleApplication3	test.c	5	
C2371	'c_to_f': 재정의. 기본 형식이 다릅니다.	ConsoleApplication3	test.c	9	



함수 원형

15

- 함수 원형(function prototyping): 컴파일러에게 함수에 대하여 미리 알리는 것



```
#include <stdio.h>
double c_to_f(double c_temp); // 함수 원형

int main(void)
{
    printf("섭씨 %lf도는 화씨 %lf입니다. \n", 36.0, c_to_f(36.0));
    return 0;
}
double c_to_f(double c_temp)
{
    return 9.0 / 5.0 * c_temp + 32;
}
```



함수 원형

16

- 함수 원형은 함수의 이름, 매개변수, 반환형을 함수가 정의되기 전에 미리 알려주는 것이다.
- 함수 원형은 함수 헤더에 세미콜론(;)만을 추가한 것과 똑같다. 다만 함수 원형에서는 매개 변수의 이름은 적지 않아도 된다. 매개 변수의 자료형만 적으면 된다.

```
double c_to_f(double);  
int get_integer(void);
```

매개 변수의 이름은 생략하여도 된다.
반드시 끝에 ;을 붙여야 한다.



함수 원형을 사용하지 않는 예제

17

```
int compute_sum(int n)
{
    int i;
    int result = 0;
    for(i = 1; i <= n; i++)
        result += i;
    return result;
}
```

```
int main(void)
{
    int sum;
    sum = compute_sum(100);    printf("sum=%d \n", sum);
}
```



함수 정의가 함수 호출보다 먼저 오면 함수 원형을 정의하지 않아도 된다.

그러나 일반적인 방법은 아니다.



함수 원형을 사용하지 않는 예제

18

```
#include <stdio.h>
double sub1(double d)
{
    sub2(100.0);
}
double sub2(double d)
{
    sub1(20.0);
}
int main(void)
{
    return 0;
}
```

이런 경우에는 원형 말고는 방법이 없음

오류 목록						
전체 솔루션		1 오류	1 경고	0 메시지	빌드 + IntelliSense	검색 오류 목록
코드	설명	프로젝트	파일	줄	비표시 오류(Suppr...)	
C4013	'sub2'이(가) 정의되지 않았습니다. extern은 int형을 반환하는 것으로 간주합니다.	Project14	소스.c	4		
C2371	'sub2': 재정의. 기본 형식이 다릅니다.	Project14	소스.c	6		



참고

19

★ 참고사항

링크 오류

소스 파일을 컴파일하여서 실행 파일을 만들다 보면 링크 오류가 발생하는 경우가 있다. 링크 오류는 컴파일러가 함수를 찾지 못했을 때 발생한다. 즉 프로그래머가 정의되지 않은 함수를 사용하였을 때 링크 오류가 발생한다. 링크 오류가 발생하면 함수의 이름이 오류 메시지로 표시된다. 따라서 오류 메시지를 보고 함수가 확실히 정의되었는지를 확인하여야 한다.

```
1>c:\users\chun\documents\visual studio 2010\projects\hello\hello\hello.c(4): warning C4013:  
'sub'이(가) 정의되지 않았습니니다. extern은 int형을 반환하는 것으로 간주합니다.
```

```
1>hello.obj : error LNK2001: _sub 외부 기호를 확인할 수 없습니다.
```

```
1>C:\Users\chun\documents\visual studio 2010\Projects\hello\Debug\hello.exe : fatal error  
LNK1120: 1개의 확인할 수 없는 외부 참조입니다.
```

```
===== 빌드: 성공 0, 실패 1, 최신 0, 생략 0 =====
```

링크 오류



라이브러리 함수

20

- 라이브러리 함수(library function): 컴파일러에서 제공하는 함수
 - 표준 입출력
 - 수학 연산
 - 문자열 처리
 - 시간 처리
 - 오류 처리
 - 데이터 검색과 정렬





난수 함수

21

- 난수(**random number**)는 규칙성이 없이 임의로 생성되는 수이다.
- 난수는 암호학이나 시뮬레이션, 게임 등에서 필수적이다.
- rand()
 - 난수를 생성하는 함수
 - 0부터 RAND_MAX까지의 난수를 생성

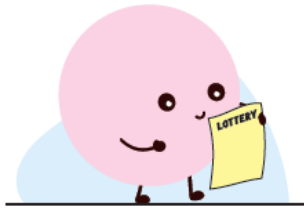




예제: 로또 번호 생성하기

22

- 하나의 예제로 로또 번호를 생성하는 프로그램을 작성해보자. 로또 번호는 1부터 45까지의 숫자 6개로 이루어진다.





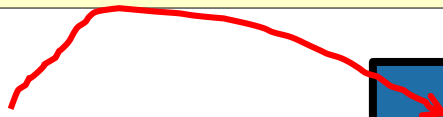
실습 코드

23

```
#include <stdio.h>
#include <stdlib.h>
int main(void)
{
    int i;
    for(i = 0; i < 6; i++)
        printf("%d ", rand());

    return 0;
}
```

0에서 32767 사이의 정수로 생성



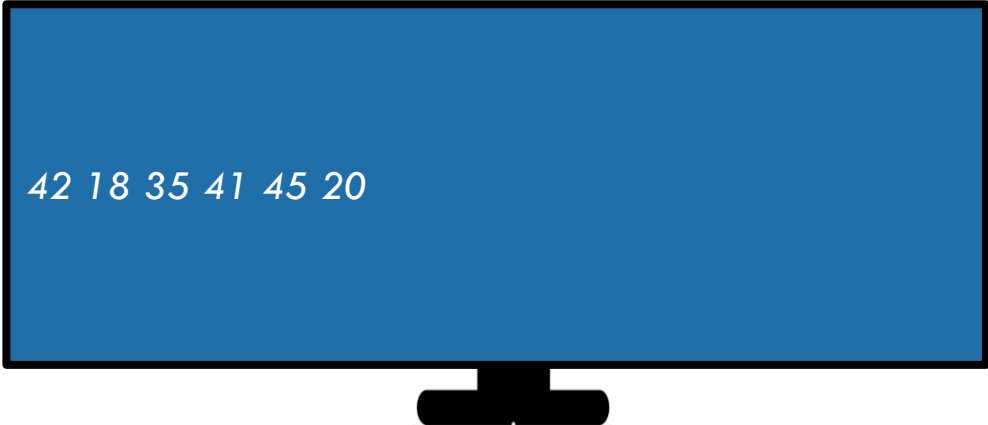
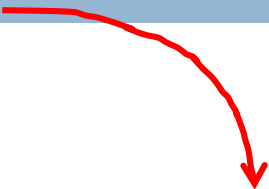
41 18467 6334 26500 19169 15724



1 부터 45 사이로 제한

24

- `printf("%d ", 1+(rand()%45));`



42 18 35 41 45 20

- 하지만 실행할 때마다 항상 똑같은 난수가 발생된다.



실행할 때마다 다르게 하려면

25

- 매번 난수를 다르게 생성하려면 시드(**seed**)를 다르게 하여야 한다.
 - `srand( (unsigned)time( NULL ) );`

```
#include <stdlib.h>
#include <stdio.h>
#include <time.h>
```

```
#define MAX 45
```

```
int main( void )
```

```
{
```

```
    int i;
```

```
    srand( (unsigned)time( NULL ) );
```

```
    for( i = 0; i < 6; i++ )
```

```
        printf("%d ", 1+rand()%MAX );
```

```
    return 0;
```

```
}
```

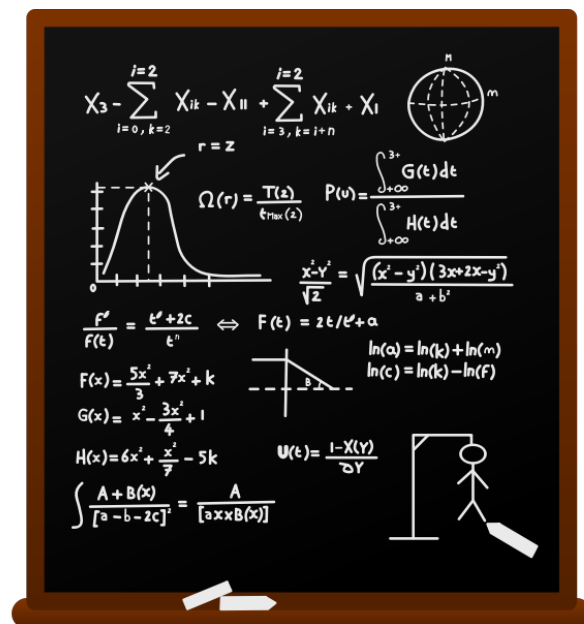
시드를 설정하는 가장 일반적인 방법은 현재의 시각을 시드로 사용하는 것이다. 현재 시각은 실행할 때마다 달라지기 때문이다.



표준 라이브러리 함수(수학 함수)

26

- 이번 장에서는 수치 계산을 하는 라이브러리 함수에 대하여 살펴본다. 이들 라이브러리 함수를 사용하면 복잡한 산술 연산을 할 수 있다.
- 수학 함수들에 대한 원형은 헤더파일 **math.h**에 있다. 수학 함수는 일반적으로 **double**형의 매개 변수와 반환값을 가진다.





수학 라이브러리 함수

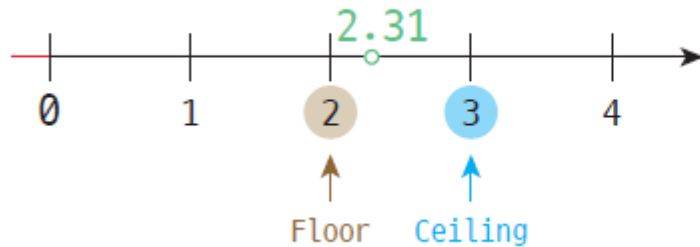
27

분류	함수	설명
삼각함수	<code>double sin(double x)</code>	사인값 계산
	<code>double cos(double x)</code>	코사인값 계산
	<code>double tan(double x)</code>	탄젠트값 계산
역삼각함수	<code>double acos(double x)</code>	역코사인값 계산 결과값 범위 $[0, \pi]$
	<code>double asin(double x)</code>	역사인값 계산 결과값 범위 $[-\pi/2, \pi]$
	<code>double atan(double x)</code>	역탄젠트값 계산 결과값 범위 $[-\pi/2, \pi]$
쌍곡선함수	<code>double cosh(double x)</code>	쌍곡선 코사인
	<code>double sinh(double x)</code>	쌍곡선 사인
	<code>double tanh(double x)</code>	쌍곡선 탄젠트
지수함수	<code>double exp(double x)</code>	e^x
	<code>double log(double x)</code>	$\log_e x$
	<code>double log10(double x)</code>	$\log_{10} x$
기타함수	<code>double ceil(double x)</code>	x보다 작지 않은 가장 작은 정수
	<code>double floor(double x)</code>	x보다 크지 않은 가장 큰 정수
	<code>double fabs(double x)</code>	실수 x의 절대값
	<code>int abs(int x)</code>	정수 x의 절대값
	<code>double pow(double x, double y)</code>	x^y
	<code>double sqrt(double x)</code>	$\sqrt{x}$



floor()와 ceil() 함수

28



floor(x)



ceil(x)

```
double result, value = 1.6;
```

```
result = floor(value);           // result는 1.0이다.  
printf("%lf ", result);
```

```
result = ceil(value);           // result는 2.0이다.  
printf("%lf ", result);
```



fabs()

29

- fabs()는 실수를 받아서 절대값을 반환한다.

```
printf("12.0의 절대값은 %f\n", fabs(12.0));  
printf("-12.0의 절대값은 %f\n", fabs(-12.0));
```



pow()와 sqrt()

30

```
printf("10의 3승은 %.0f.\n", pow(10.0, 3.0));  
printf("16의 제곱근은 %.0f.\n", sqrt(64));
```

*10의 3승은 1000.
16의 제곱근은 4.*



기타 함수

31

함수	설명
<code>exit()</code>	<code>exit()</code> 를 호출하면, 실행 중인 프로그램을 종료시킨다.
<code>system("...")</code>	<code>system()</code> 은 운영 체제의 명령 프롬프트에게 명령어를 전달하여서 실행시키는 함수이다. 예를 들어서 DOS 명령어인 DIR이나 COPY, TYPE, CLS, DEL, MKDIR와 같은 명령어들을 실행시킬 수 있다.
<code>time(NULL)</code>	현재 시각을 반환한다. 1970년 1월 1일부터 흘러온 초를 반환한다.



복잡한 프로그램은 함수로 분리한다.

32

```
int main(void)
{
    // 숫자들의 리스트를 키보드에서 읽어들이는 코드
    ...
    // 숫자들을 크기순으로 정렬하는 코드
    ...
    // 정렬된 숫자들의 리스트를 화면에 출력하는 코드
    ...
}
```



```
int main(void)
{
    ...
    read_list(); // 숫자들의 리스트를 키보드에서 읽어 들이는 함수
    sort_list(); // 숫자들의 리스트를 크기순으로 정렬하는 함수
    print_list(); // 숫자들의 리스트를 화면에 출력하는 함수
    ...
}
```


C 언어 EXPRESS(개정판)



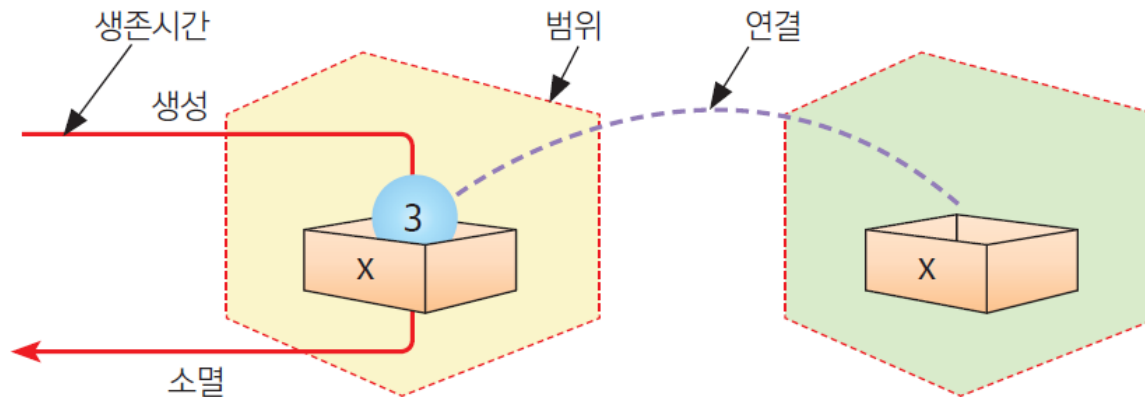
제 9장 함수와 변수



변수의 속성

34

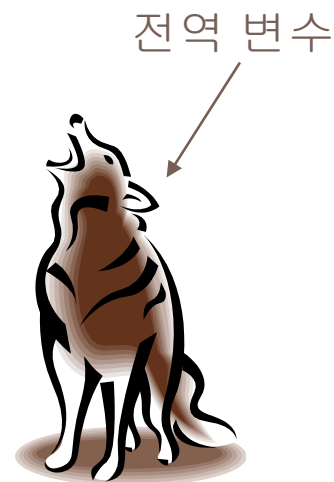
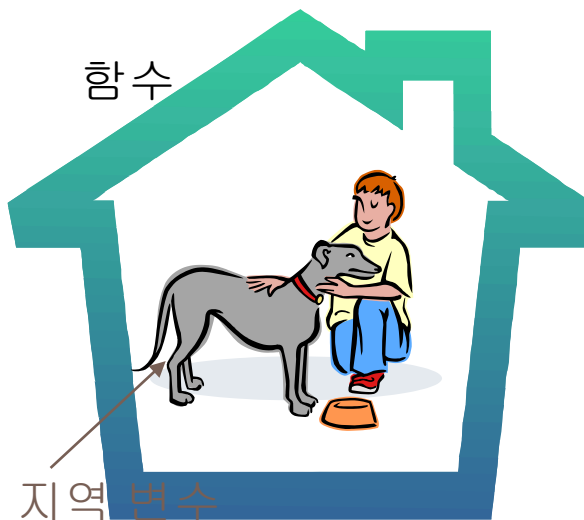
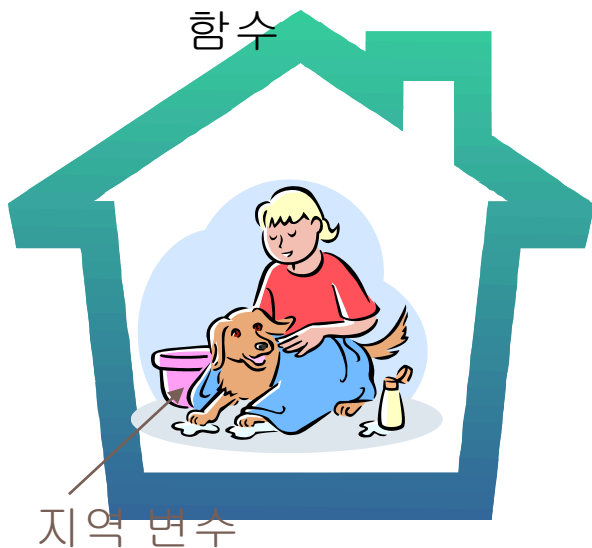
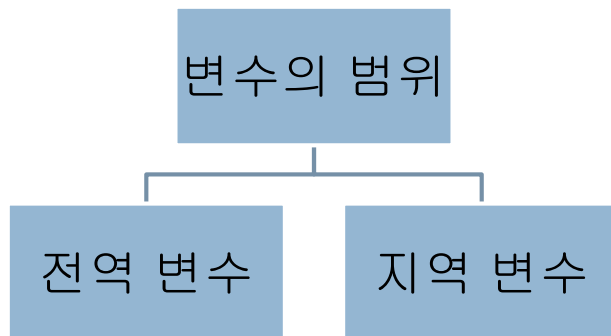
- 변수의 속성 : 이름, 타입, 크기, 값 + 범위, 생존 시간, 연결
 - 범위(scope) : 변수가 사용 가능한 범위, 가시성
 - 생존 시간(lifetime) : 메모리에 존재하는 시간
 - 연결(linkage) : 다른 영역에 있는 변수와의 연결 상태





변수의 범위

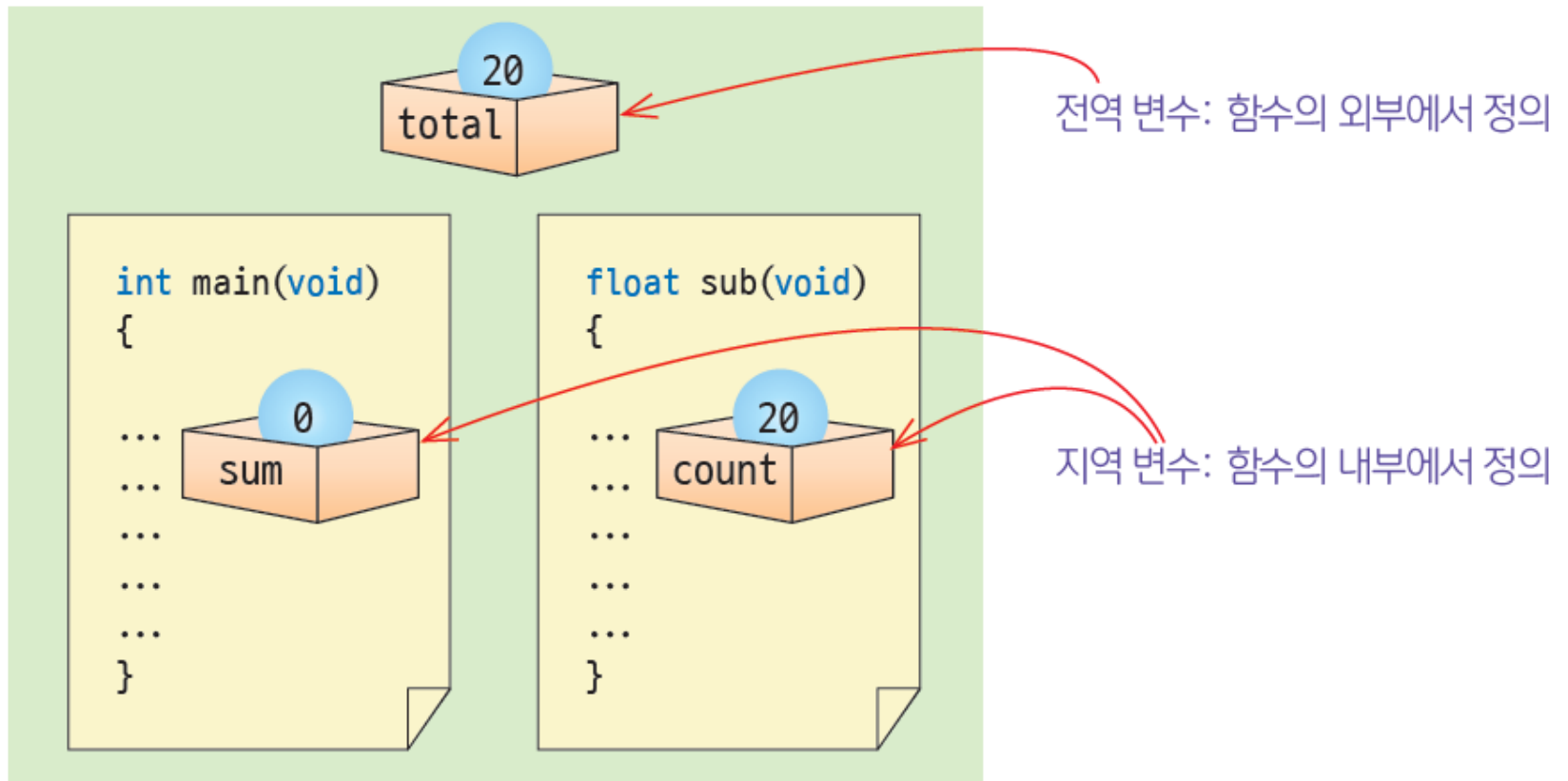
35





전역 변수와 지역 변수

36





지역 변수

37

- 지역 변수(local variable)는 블록 안에 선언되는 변수

```
int sub(void)
{
    int x = 0;

    while(flag!= 0){
        int y;
        ...
    }

    y = 0; // 오류!!
    ...
}
```

지역 변수 x가 사용 가능한 범위

지역 변수 y가 사용 가능한 범위

y가 선언된 블록을 벗어나서 사용하였으므로 오류!

지역 변수는 선언된 블록을 떠나면 안됩니다.





지역 변수 선언 위치

38

- 최신 버전의 C에서는 블록 안의 어떤 위치에서도 선언 가능!!

```
while(1) {  
    ...  
    ...  
    int sum = 0;  
    ...  
}
```

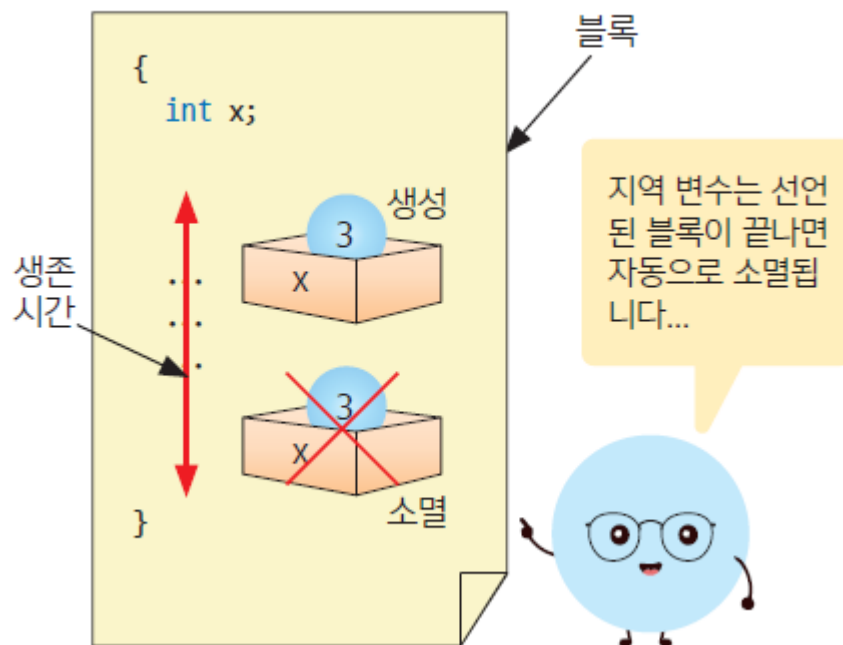
블록의 중간에서도 얼마든지 지역 변수
를 선언할 수 있다.





지역 변수의 생존 기간

40





지역 변수 예제

41

```
#include <stdio.h>

int main(void)
{
    int i;

    for(i = 0; i < 5; i++)
    {
        int temp = 1;
        printf("temp = %d\n", temp);
        temp++;
    }
    return 0;
}
```

블록이 시작할 때 마다
생성되어 초기화된다.

```
temp = 1
temp = 1
temp = 1
temp = 1
temp = 1
```

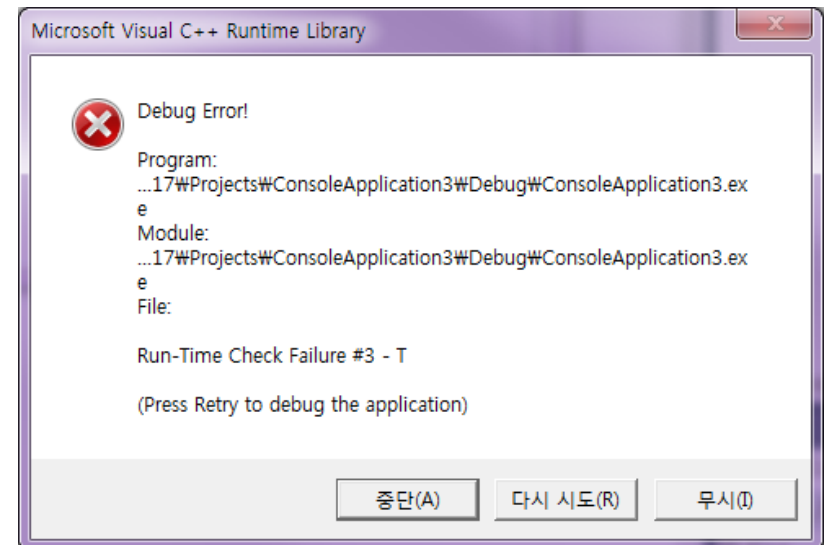


지역 변수의 초기값

42

```
#include <stdio.h>
int main(void)
{
    int temp;
    printf("temp = %d\n", temp);
    return 0;
}
```

초기화 되지
않았으므로 쓰레기
값을 가진다.





함수의 매개 변수

43

- 함수의 헤더 부분에 정의되어 있는 매개 변수도 일종의 지역 변수이다. 즉 지역 변수가 지니는 모든 특징을 가지고 있다.
- 지역 변수와 다른 점은 함수 호출시의 인수 값으로 초기화되어 있다는 점이다.

```
int inc(int counter)
{
    counter++;
    return counter;
}
```

매개 변수도 일종의
지역 변수



함수의 매개 변수

44

```
#include <stdio.h>
int inc(int counter);
```

```
int main(void)
{
    int i;
```

```
    i = 10;
    printf("함수 호출전 i=%d\n", i);
    inc(i);
    printf("함수 호출후 i=%d\n", i);
    return 0;
```

```
}
void inc(int counter)
{
    counter++;
}
```

값에 의한 호출
(call by value)

매개 변수도 일종의
지역 변수임

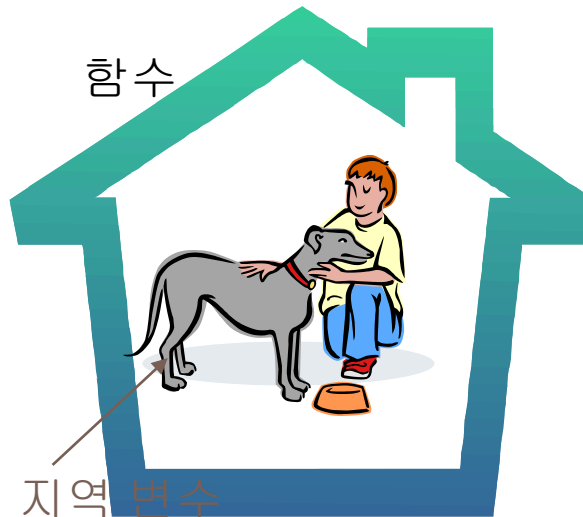
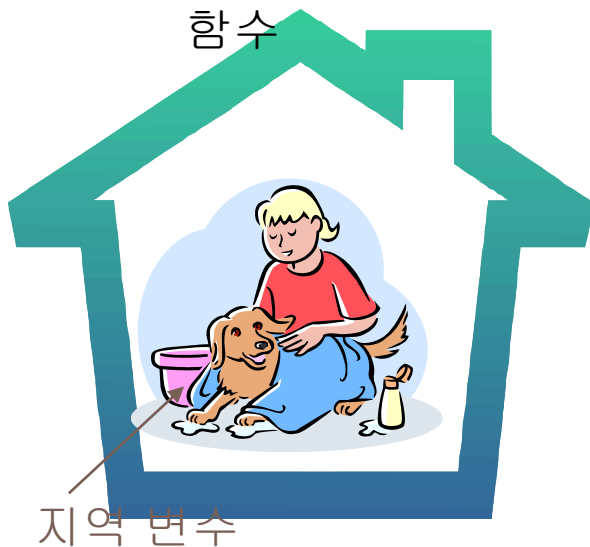
함수 호출전 i=10
함수 호출후 i=10



전역 변수

45

- 전역 변수(global variable)는 함수 외부에서 선언되는 변수이다.
- 전역 변수의 범위는 소스 파일 전체이다.





전역 변수의 초기값과 생성 기간

46

```
#include<stdio.h>
```

```
int A;
```

```
int B;
```

```
int add()
```

```
{
```

```
    return A + B;
```

```
}
```

```
int main(void)
```

```
{
```

```
    int answer;
```

```
    A = 5;
```

```
    B = 7;
```

```
    answer = add();
```

```
    printf(" % d + % d = % d\n", A, B, answer);
```

```
    return 0;
```

```
}
```

전역 변수
초기값은 0

5 + 7 = 12

전역
변수의
범위



전역 변수의 초기화

47

```
#include <stdio.h>
```

```
int counter;
```

```
int main(void)
```

```
{
```

```
    printf("counter = % d\n", counter);
```

```
    return 0;
```

```
}
```

전역 변수는 컴파일러가 프로그램 실행시에
0으로 초기화한다.

counter = 0



전역 변수의 사용

48

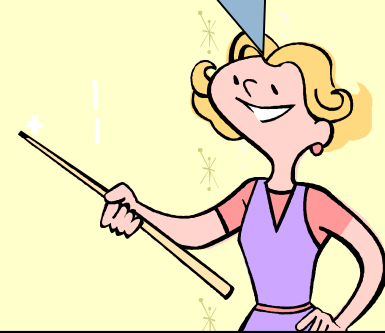
```
#include <stdio.h>
```

```
int x;  
void sub();
```

```
int main(void)  
{  
    for (x = 0; x < 10; x++)  
        sub();  
}
```

```
void sub()  
{  
    for (x = 0; x < 10; x++)  
        printf("*");  
}
```

출력은
어떻게
될까요?





전역 변수의 사용

49

- 거의 모든 함수에서 사용하는 공통적인 데이터는 전역 변수로 한다.
- 일부의 함수들만 사용하는 데이터는 전역 변수로 하지 말고 함수의 인수로 전달한다.



같은 이름의 전역 변수와 지역 변수

50

```
#include <stdio.h>
```

```
int sum = 1; // 전역 변수
```

```
int main(void)
```

```
{
```

```
    int sum = 0; // 지역 변수
```

```
    printf("sum = %d\n", sum);
```

```
    return 0;
```

```
}
```

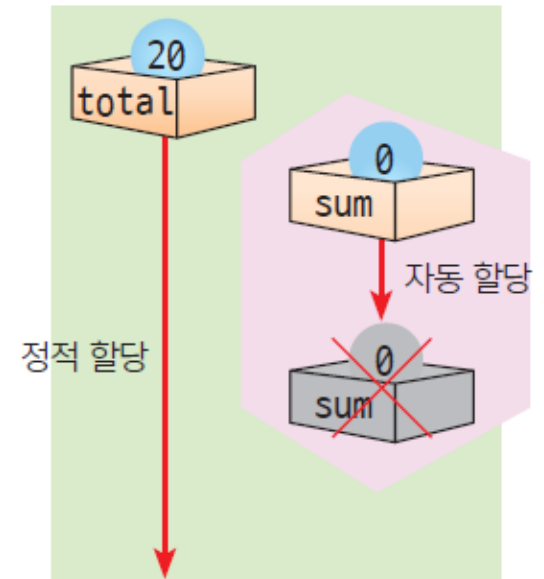
전역 변수와 지역 변수가 동일한 이름으로 선언된다.

sum = 0



- 정적 할당(static allocation):
 - 프로그램 실행 시간 동안 계속 유지
- 자동 할당(automatic allocation):
 - 블록에 들어갈 때 생성
 - 블록에서 나올 때 소멸

정적 할당은 변수가 실행 시간
내내 존재하지만 자동 할당은
블록이 종료되면 소멸됩니다.





생존 기간

52

- 생존 기간을 결정하는 요인
 - 변수가 선언된 위치
 - 저장 유형 지정자
- 저장 유형 지정자
 - auto
 - register
 - static
 - extern



저장 유형 지정자 auto

53

- 변수를 선언한 위치에서 자동으로 만들어지고 블록을 벗어나게 되며 자동으로 소멸되는 저장 유형을 지정
- 지역 변수는 **auto**가 생략되어도 자동 변수가 된다.

```
int main(void)
```

```
{
```

```
    auto int sum = 0;
```

```
    int i = 0;
```

```
    ...
```

```
    ...
```

```
}
```

전부 자동 변수로서 함수가
시작되면 생성되고 끝나면
소멸된다.



저장 유형 지정자 static

54

```
#include <stdio.h>
```

```
void sub() {
```

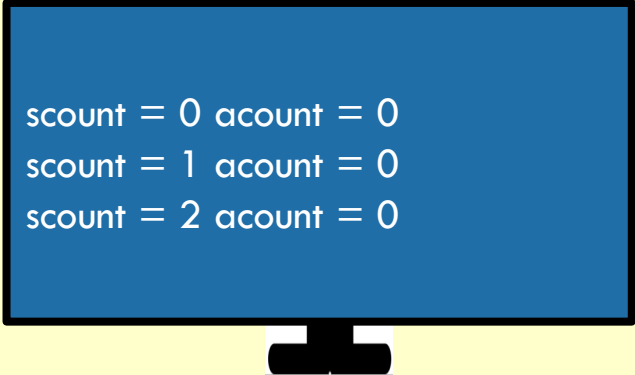
```
    static int scout = 0;  
    int account = 0;  
    printf("scout = %d\t", scout);
```

```
    printf("account = %d\n", account);  
    scout++;  
    account++;
```

```
}
```

```
int main(void) {  
    sub();  
    sub();  
    sub();  
    return 0;  
}
```

정적 지역 변수로서 static을 붙이면
지역 변수가 정적 변수로 된다.



```
scout = 0 account = 0  
scout = 1 account = 0  
scout = 2 account = 0
```



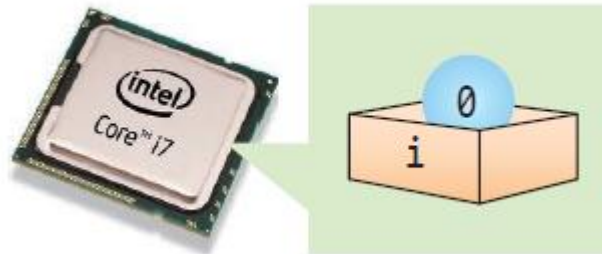
저장 유형 지정자 register

55

- 레지스터(register)에 변수를 저장.

```
register int i;  
for(i = 0; i < 100; i++)  
    sum += i;
```

CPU안의 레지스터에
변수가 저장됨





Lab: 은행 계좌 구현하기

56

- 돈만 생기면 저금하는 사람을 가정하자. 이 사람을 위한 함수 `save(int amount)`를 작성하여 보자. 이 함수는 저금할 금액을 나타내는 인수 `amount`만을 받으며 `save(100)`과 같이 호출된다. `save()`는 정적 변수를 사용하여 현재까지 저축된 총액을 기억하고 있으며 한번 호출될 때마다 총 저축액을 다음과 같이 화면에 출력한다.

```
=====
입금   출금   잔고
=====
10000           10000
50000           60000
          10000  50000
30000           80000
=====
```




```
#include <stdio.h>
```

```
// amount가 양수이면 입금이고 음수이면 출금으로 생각한다.
```

```
void save(int amount)
```

```
{
```

```
    static long balance = 0;
```

```
    if (amount >= 0)
```

```
        printf("%d \t\t", amount);
```

```
    else
```

```
        printf("\t %d \t", -amount);
```

```
    balance += amount;
```

```
    printf("%d \n", balance);
```

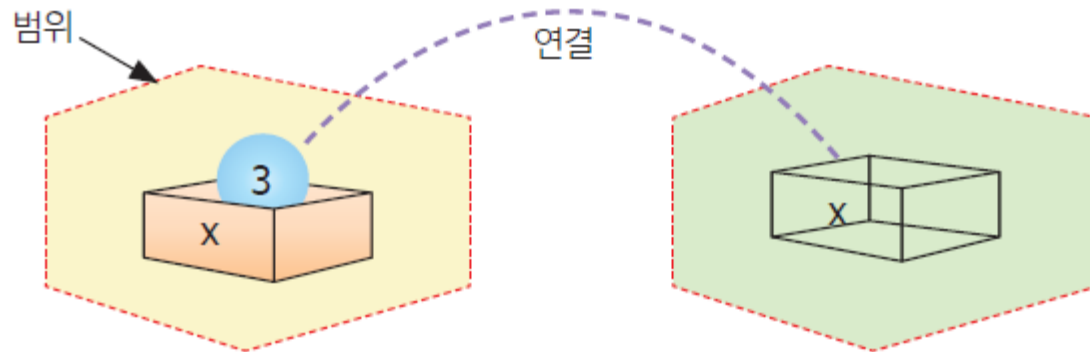
```
}
```



```
int main(void) {  
    printf("=====\n");  
    printf("입금 \t출금\t 잔고\n");  
    printf("=====\n");  
    save(10000);  
    save(50000);  
    save(-10000);  
    save(30000);  
    printf("=====\n");  
    return 0;  
}
```



- *연결(linkage)*: 다른 범위에 속하는 변수들을 서로 연결하는 것
 - 외부 연결
 - 내부 연결
 - 무연결
- 전역 변수만이 연결을 가질 수 있다.

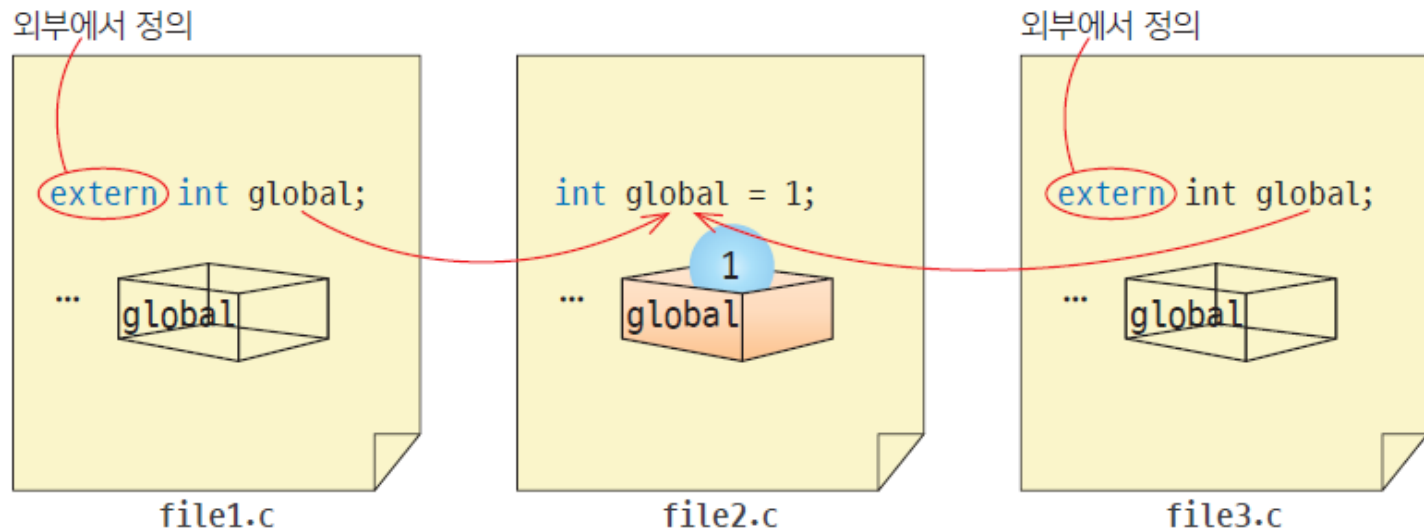




외부 연결

60

- 전역 변수를 `extern`을 이용하여서 서로 연결





연결 예제

61

- 전역 변수 또는 함수 앞에 **static**이 붙으면 내부연결이 된다. 즉, 하나의 소스 파일에서만 해당 변수나 함수를 사용할 수 있다.

```
linkage1.c

#include <stdio.h>

int all_files;
static int this_file;
extern void sub();

int main(void)
{
    sub();
    printf("%d\n", all_files);
    return 0;
}
```

```
linkage2.c

extern int all_files;

// extern int this_file;

void sub(void)
{
    all_files = 10;
}
```



함수의 static

62

main.c

```
#include <stdio.h>
```

```
//extern void f1();
```

```
extern void f2();
```

```
int main(void)
```

```
{
```

```
    f2();
```

```
    return 0;
```

```
}
```

sub.c

```
#include <stdio.h>
```

```
static void f1()
```

```
{
```

```
    printf("f1()가 호출되었습니다.\n");
```

```
}
```

```
void f2()
```

```
{
```

```
    printf("f2()가 호출되었습니다.\n");
```

```
}
```

f2()가 호출되었습니다.



블록에서 extern을 이용한 전역 변수 참조

63

- extern은 블록에서 전역 변수에 접근할 때도 사용된다.

```
#include <stdio.h>
int x = 50;

int main(void)
{
    int x = 100;
    {
        extern int x;
        printf("x= %d\n", x);
    }
    return 0;
}
```

x= 50



어떤 저장 유형을 사용해야 하는가?

64

- 일반적으로는 *자동 저장 유형* 사용 권장
- 변수의 값이 함수 호출이 끝나도 그 값을 유지하여야 할 필요가 있다면 *지역 정적*
- 만약 많은 소스 파일에서 공유되어야 하는 변수라면 *외부 참조 변수*

저장 유형	키워드	정의되는 위치	범위	생존 시간
자동	auto	함수 내부	지역	임시
레지스터	register	함수 내부	지역	임시
정적 지역	static	함수 내부	지역	영구
전역	없음	함수 외부	모든 소스 파일	영구
정적 전역	static	함수 외부	하나의 소스 파일	영구
외부 참조	extern	함수 외부	모든 소스 파일	영구



순환(recursion)이란?

65

- 함수는 자기 자신을 호출할 수도 있다. 이것을 순환(recursion)라고 부른다.

$$n! = \begin{cases} 1 & n=0 \\ n*(n-1)! & n \geq 1 \end{cases}$$

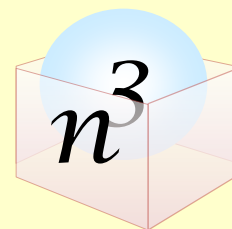


팩토리얼 구하기

66

- 팩토리얼 프로그래밍: $(n-1)!$ 팩토리얼을 현재 작성중인 함수를 다시 호출하여 계산(순환 호출)

```
int factorial(int n)
{
    if( n <= 1 ) return(1);
    else return (n * factorial(n-1) );
}
```

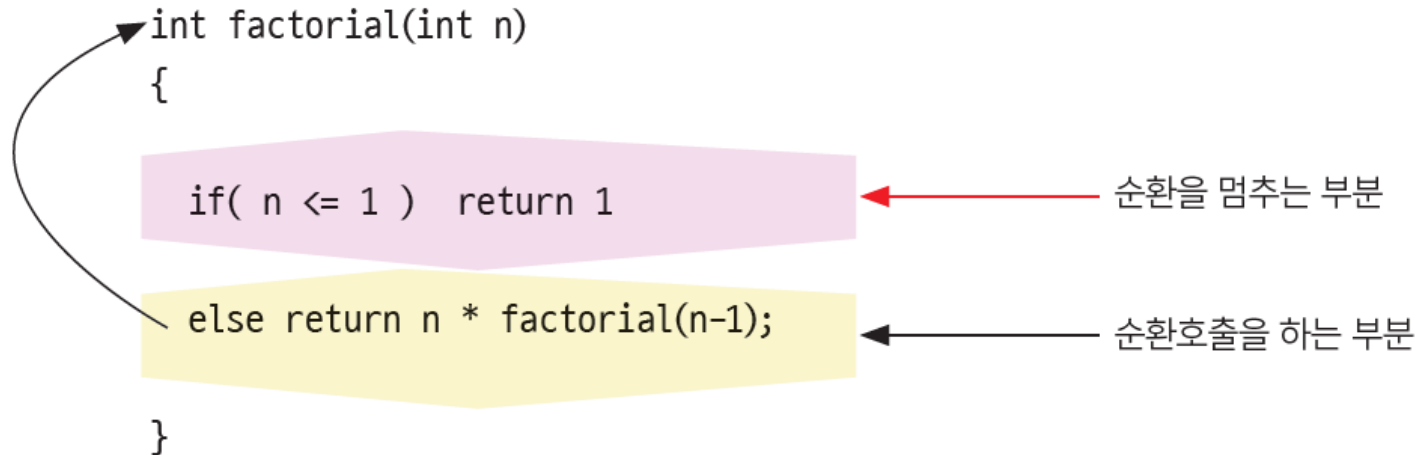




순환 함수의 구조

67

- 순환 알고리즘은 그림 9-9와 같이 자기 자신을 순환적으로 호출하는 부분과 순환 호출을 멈추는 부분으로 구성되어 있다.



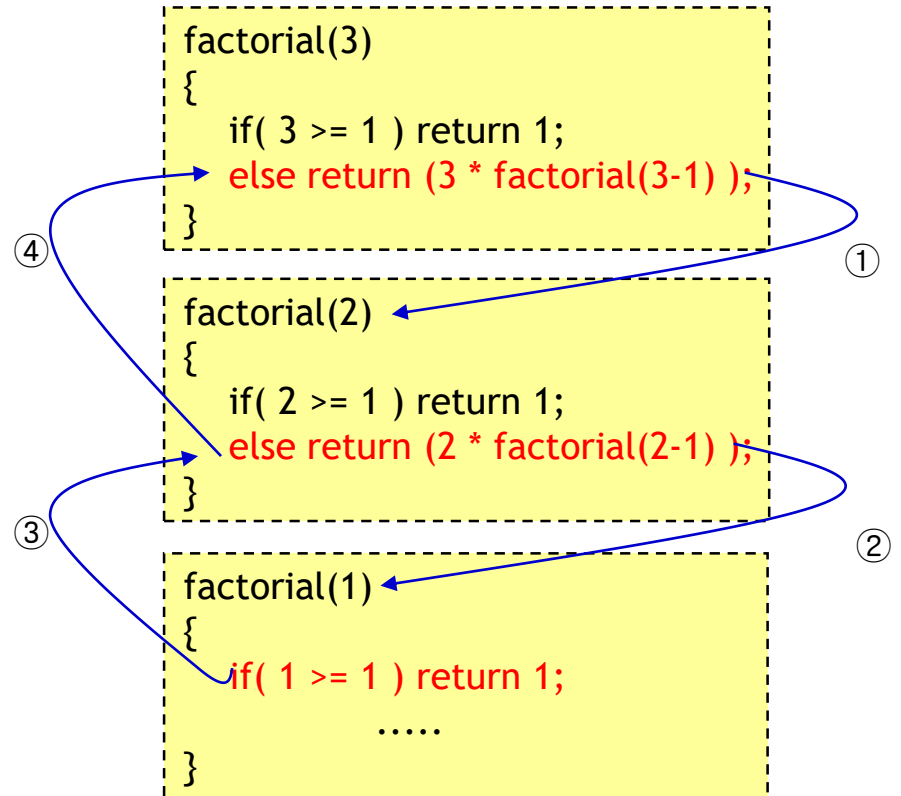


팩토리얼 구하기

68

- 팩토리얼의 호출 순서

factorial(3) = 3 * factorial(2)
= 3 * 2 * factorial(1)
= 3 * 2 * 1
= 3 * 2
= 6





팩토리얼 계산

69

// 재귀적인 팩토리얼 함수 계산

```
#include <stdio.h>
```

```
long factorial(int n)
```

```
{
```

```
    printf("factorial(%d)\n", n);
```

```
    if (n <= 1) return 1;
```

```
    else return n * factorial(n - 1);
```

```
}
```

```
int main(void)
```

```
{
```

```
    int x = 0;
```

```
    long f;
```

```
    printf("정수를 입력하시오:");
```

```
    scanf(" %d", &n);
```

```
    printf("%d!은 %d입니다. \n", n, factorial(n));
```

```
    return 0;
```

```
}
```

정수를 입력하시오:5

factorial(5)

factorial(4)

factorial(3)

factorial(2)

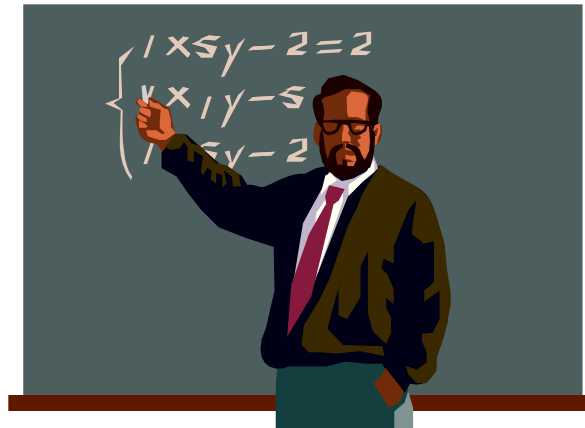
factorial(1)

5!은 120입니다.



Q & A

70





Q & A

71

